

Fundamentals of Data Science

Lecture 5 - Feature Engineering

Adrian Ochs ¹ and Christian Roerig ²

Michaelmas 2025

¹Co-founder @ Quantum Leap, ex-QuantCo

²Data Scientist/ML Engineer @ QuantCo

Table of contents

1. The two scopes of feature engineering
2. A note on data leakage
3. Numerical features
4. Categorical features
5. Missings
6. Outliers
7. Dimension reduction
8. Extracting features from text
9. Feature Transformation Pipelines

The two scopes of feature engineering



The two scopes of feature engineering

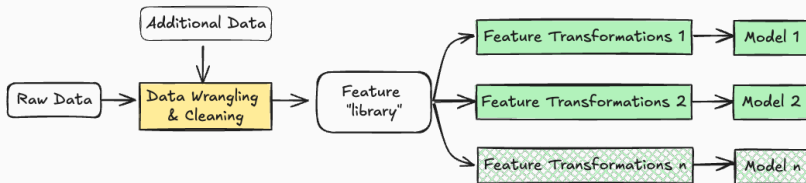
Feature engineering can be typically split up in two parts:

- **Preparing feature library:** This spans all activities from fixing dtypes, cleaning the data to creating new features from existing or extra sources and documenting features. Note however, that **these activities usually are observation/row-specific**.
- **Transforming features for modelling:** This stage transforms the features to be used by the estimator. **Transformations can be model-specific and can use information from multiple rows**, e.g. mean imputation or quantile bins. We have to be careful about potential data leakage between train and validation sets.

Note

We will mostly focus on the second part here, but finding good predictive features in the first place is key.

The two scopes of feature engineering



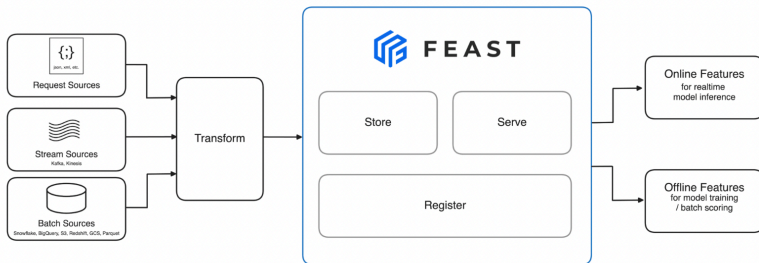
Note

Most textbook examples do not distinguish here as they usually present a fixed feature set for a given modelling task and any extra flexibility would introduce overhead. In real-world settings we usually benefit from more flexibility.

Feature library - Sharing your feature specifications for modelling and production

It's good practice to organize all your feature definitions in a central place. This allows you to share your feature definitions between modelling and production and serves as a central store and documentation for features for various use cases.

Here is an illustration by the open source feature store library [feast](#).



Feature library - Domain knowledge is key

For one use case it's perfectly fine to organize your feature preparation in a function or as part of a module (that's what we'll stick to throughout this course), but as more use cases and users appear it makes sense to think about some central feature repository.

Domain knowledge is key

Coming up with predictive features for the given use case is key. Leveraging domain knowledge is essential here to come up with impactful features. Also combinations of existing features such as ratios (e.g. horse powers per kg for used car prices, clicks per second for bot detection) can already provide more condensed information for a model.

Transforming features for modelling

Feature transformations should relate to the specific estimator:

- Can the estimator cope with missings or do we want or have to impute them?
- Can the estimator cope with categoricals or should we one-hot-encode them?
- Do we have to parameterize the model (e.g. define interaction terms) or do we use a non-parametric estimator?
- ...

Example

A parametric model might need non-linear transformations of numeric features to capture non-linear effects in the data, while approaches using decision trees as learners (e.g. Random Forest or Gradient Boosted Trees) will find adequate splits.

Transforming features for modelling

Feature transformations might also address the intended model behaviour:

- What degree of functional flexibility do we need/want?
- Do we want continuous changes in predictions vs. discrete (e.g. splines vs. bins)?
- What category should we fall back to in case of unseen values?
- ...

A note on data leakage



What is data leakage?

In the context of statistical learning or machine learning, data leakage refers to a scenario where information from outside the training dataset is inappropriately used to create a model. This can lead to the model performing unrealistically well on the training data but failing to generalize to new, unseen data accurately.

How can data leakage occur?

Data leakage in statistical learning can occur in various ways, including:

- **Leaking Information from the Future:** Using information that would not actually be available at the time of prediction, such as including future data in the training set.
- **Target Leakage:** Inclusion of features that directly or indirectly leak information about the target variable, leading to artificially inflated model performance.
- **Data Preprocessing:** Inappropriate data preprocessing steps that introduce information from the test set into the training set, affecting model performance.

Examples

- **Leaking Information from the Future:** Training a model for stock price prediction but including future stock prices.
- **Target Leakage:** Say you want to predict conversion of customers on your website, but include clicks on "Buy" button as feature.
- **Data Preprocessing:** Consider a dataset with missing values in some columns. If the missing values are imputed using information that is only available in the test set, the model may perform seemingly well in training but poorly on completely new data.

Avoid leakage using clear train/test splits

Feature transformations or feature preprocessing should be treated as the first stage of your model, as we will already be using information from multiple observations when binning, imputing etc.

In fact, our model pipeline will be more than just an estimator but also contain model specific feature transformations informed by the data.

Fix your train and validation set definition

It's good practice to fix your train and validation set definition for any model pipeline we are training. Otherwise, when choosing random sample splits, it is virtually impossible to keep track which data has been seen by the specific model pipelines which makes comparisons likely flawed by data leakage.

Tip: Use some hash based on some primary keys, so the same contract/person/... always ends up in the same train/test bucket.

Numerical features



Feature Engineering of numerical features

There are various ways to transform numerical features for modelling. Typically we should ask us some of the following questions:

- Should the feature be discretized or do we keep it at some continuous granularity (e.g. binning age in age groups or using it at a daily/monthly/yearly frequency)?
- Do we need to scale the feature?
- Do we need to add extra transformations of the feature to capture non-linearities (e.g. polynomials, splines)?

Note

Some answers do heavily depend on the underlying estimator. For instance, when using a parametric model we have to think about which extra terms to add to capture potential non-linearities, but when using e.g. tree-based approaches such as Random Forests or GBMs, the trees take care of this.

Feature Engineering of numerical features

In the following, we'll go over several transformation types for numerical features and add some recommendations when to use them.

In particular, we'll cover the following transformers:

- **Binning:** Quantile binning, Custom binning.
- **Scaling:** Standardization, Scale to range.
- **Non-linear transformations:** Log-transform, Polynomials, Splines.

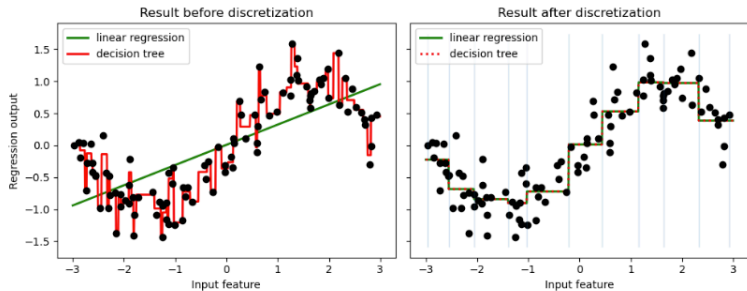
Note

We will cover transformations to cope with missings and outliers in the respective sections.

Binning of numerical features

- Discretizing numerical features is one way to allow for relatively flexible marginal effects in parametric models, as we estimate a coefficient for each of the $n-1$ (n if no intercept is included) bins.
- It might also be the case that you want your policy function (e.g. price function) to return the same marginal effect for some groups, e.g. age groups.

Binning adds flexibility to parametric models



Source: [scikit-learn docs](#)

Quantile Binning

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import KBinsDiscretizer

# Generating example data for ages
np.random.seed(0)
df = pd.DataFrame({"age": np.random.randint(18, 80, 100)})

# Creating KBinsDiscretizer object with quantile strategy
est = KBinsDiscretizer(n_bins=5, strategy='quantile')

# Fitting and transforming the ages data
dft = est.fit_transform(df)

print(est.bin_edges_)

df["binned_age"] = est.inverse_transform(dft)
df
```

initiate binner using 5 bins and quantile binning

the computed bin edges are saved as part of the binner class

✓ 0.0s

[array([18. , 28.8, 40.2, 53.4, 65.4, 77.])]

	age	binned_age
0	62	59.4
1	65	59.4
2	71	71.2
3	18	23.4
4	21	23.4
...	...	...
95	71	71.2
96	30	34.5
97	75	71.2
98	60	59.4
99	38	34.5

Sometimes it might be useful to convert the data back to the original feature space. Here each value represents the mean between two bin edges.

100 rows x 2 columns

Uniform Binning

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import KBinsDiscretizer
```

```
# Generating example data for ages
np.random.seed(0)
df = pd.DataFrame({"age": np.random.randint(18, 80, 100)})
```

```
# Creating KBinsDiscretizer object with uniform strategy
est = KBinsDiscretizer(n_bins=5, strategy='uniform')
```

```
# Fitting and transforming the ages data
dft = est.fit_transform(df)
```

```
print(est.bin_edges_)
```

```
df["binned_age"] = est.inverse_transform(dft)
df
```

✓ 0.0s

```
[array([18. , 29.8, 41.6, 53.4, 65.2, 77. ])]
```

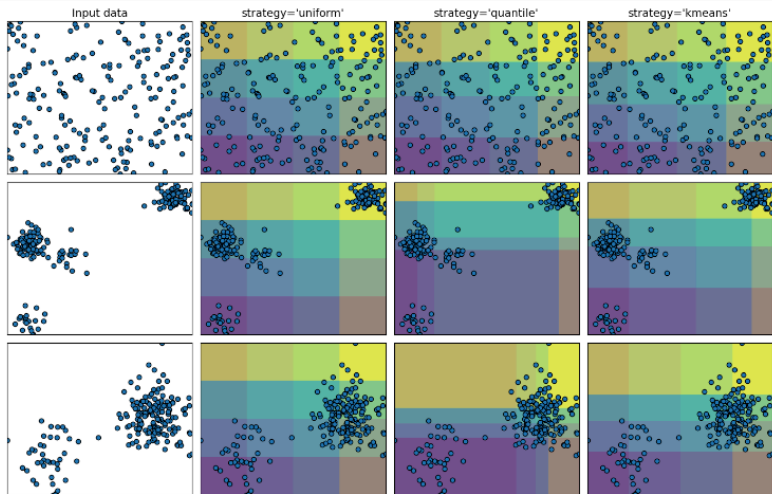
	age	binned_age
0	62	59.3
1	65	59.3
2	71	71.1
3	18	23.9
4	21	23.9
...	...	...
95	71	71.1
96	30	35.7
97	75	71.1
98	60	59.3
99	38	35.7

100 rows x 2 columns

here we can simply switch to uniform binning

it makes sense that we get almost the same bins as before as we drew the ages from a uniform distribution in the first place

Illustration of binning strategies



Source: [scikit-learn docs](#)

There might be cases in which, given the business context, specific discrete cuts are expected. Think about the mileage feature for a car insurance. This is usually given in discrete ranges, as the actual mileage is not known at the time of buying the insurance.

Note

We could either write our own transformer in this case, which we will learn to do later today, or use the flexibility of the FunctionTransformer.

Custom Binning

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import FunctionTransformer

# Generating example data for ages
np.random.seed(0)
df = pd.DataFrame({"age": np.random.randint(18, 80, 100)})

# Defining custom bins and labels
bins = [17.99, 25, 35, 60, np.inf]
labels = ['student', 'young adult', 'adult', 'senior citizen']

# Initiating FunctionTransformer with pd.cut
transformer = FunctionTransformer(
    pd.cut, kw_args={'bins': bins, 'labels': labels, 'retbins': False}
)
df["binned_age"] = transformer.fit_transform(df["age"])
df
```

define custom cutoffs
and optional labels

the FunctionTransformer allows us
to use functions of our choice
as pre-processing transformer

✓ 0.0s

	age	binned_age
0	62	senior citizen
1	65	senior citizen
2	71	senior citizen
3	18	student
4	21	student
...	...	...
95	71	senior citizen
96	30	young adult
97	75	senior citizen
98	60	adult
99	38	adult

the binning worked as expected

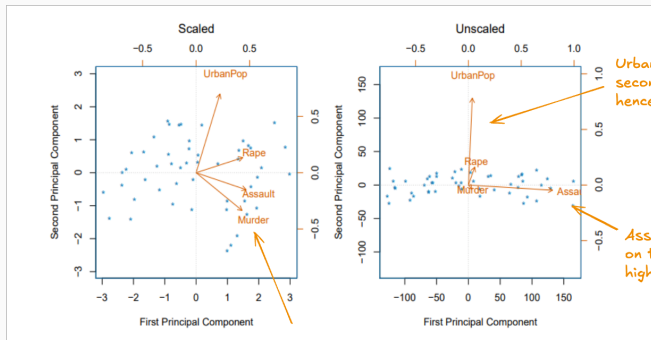
100 rows x 2 columns

Scaling of numerical features

Scikit-learn docs on scaling: Standardization of numerical variables is often good practice for various machine learning algorithms. For instance, many elements used in the objective function of a learning algorithm (such as the RBF kernel of Support Vector Machines or the l1 and l2 regularizers of linear models) may assume that all features are centered around zero or have variance in the same order. If a feature has a variance that is orders of magnitude larger than others, it might dominate the objective function and make the estimator unable to learn from other features correctly as expected. ([more details](#), [helpful medium article](#))

Illustration of scaling for PCA

As Principal Component Analysis (more on that later) aims to capture most of the variance by the least amount of dimensions, scaling is key as otherwise features with high variance dominate.



in the scaled case, the crime types contribute more equally to the first PC

Urban Population has the second largest variance, hence dominating the second PC.

Assault has the largest loading on the first PC as it has by far the highest variance.

Source: [An Introduction to Statistical Learning](#)

Standardization



```
>>> from sklearn import preprocessing
>>> import numpy as np
>>> X_train = np.array([[ 1., -1.,  2.],
...                     [ 2.,  0.,  0.],
...                     [ 0.,  1., -1.]])
>>> scaler = preprocessing.StandardScaler().fit(X_train)
>>> scaler
StandardScaler()

>>> scaler.mean_
array([1. ..., 0. ..., 0.33...])

>>> scaler.scale_
array([0.81..., 0.81..., 1.24...])

>>> X_scaled = scaler.transform(X_train)
>>> X_scaled
array([[ 0. ..., -1.22...,  1.33...],
       [ 1.22...,  0. ..., -0.26...],
       [-1.22...,  1.22..., -1.06...]])
```

Scaled data has zero mean and unit variance:

```
>>> X_scaled.mean(axis=0)
array([0., 0., 0.])

>>> X_scaled.std(axis=0)
array([1., 1., 1.])
```

Scaling to a range

```
>>> X_train = np.array([[ 1., -1.,  2.],
...                     [ 2.,  0.,  0.],
...                     [ 0.,  1., -1.]])
...
>>> min_max_scaler = preprocessing.MinMaxScaler()
>>> X_train_minmax = min_max_scaler.fit_transform(X_train)
>>> X_train_minmax
array([[0.5, 0., 1.],
       [1., 0.5, 0.33333333],
       [0., 1., 0.]])
```

The same instance of the transformer can then be applied to some new test data unseen during the fit call: the same scaling and shifting operations will be applied to be consistent with the transformation performed on the train data:

```
>>> X_test = np.array([[-3., -1.,  4.]])
>>> X_test_minmax = min_max_scaler.transform(X_test)
>>> X_test_minmax
array([-1.5, 0., 1.66666667])
```

Non-linear transformations of numerical features

For (generalized) linear models we often aim to increase the flexibility by including non-linear transformations of some numeric features.

Particularly, we will consider:

- Splines
- Polynomials

Or we might log-transform features to guard against outliers, i.e. reduce the skewness of the feature.

Log-transformation

Using the log transformation is quite common to reduce the effect of outliers or reduce the skewness to arrive at a more symmetric distribution, potentially a normal distribution if the distribution is roughly log-normal. Depending on the domain, you might need an offset (e.g. if data includes zeros).

You can simply use scikit-learn's `FunctionTransformer` to apply a log transformation.

Log-transformation

Using the log transformation is quite common to reduce the effect of outliers or reduce the skewness to arrive at a more symmetric distribution, potentially a normal distribution if the distribution is roughly log-normal. Depending on the domain, you might need an offset (e.g. if data includes zeros).

You can simply use scikit-learn's `FunctionTransformer` to apply a log transformation.

Beware Jensen's inequality

Sometimes we might want to log transform our dependent variable to arrive at a normal distribution. However, we have to be careful, when transforming it back, as Jensen's inequality taught us that the convex transformation of the expectation is less than or equal to the expectation of this convex transformation: $\varphi(E[x]) \leq E[\varphi(x)]$. There exist bias corrections (s. mean of log-normal). We'll see in the modelling chapter how we can use link functions instead.

Polynomials

In order to add polynomials, we can use scikit-learns

`PolynomialFeatures` transformer:

```
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
```

```
# Create a sample DataFrame with a numeric variable
df = pd.DataFrame({'numeric_feature': [1, 2, 3, 4, 5]})
```

```
# Initialize the PolynomialFeatures transformer with degree 3
poly = PolynomialFeatures(degree=3, include_bias=False)
```

```
# Transform the numeric variable
```

```
dft= poly.fit_transform(df[['numeric_feature']])
```

when True, this also includes
a column of 1s (power of 0)
which acts as an intercept

```
print(dft)
```

✓ 0.0s

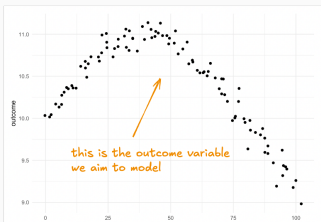
```
[[ 1.  1.  1.]
 [ 2.  4.  8.]
 [ 3.  9. 27.]
 [ 4. 16. 64.]
 [ 5. 25.125.]
```


Splines can be roughly interpreted as a mix of binning a numerical variable, but instead of indicators, we use polynomials, which adds a lot of flexibility and smoothness.

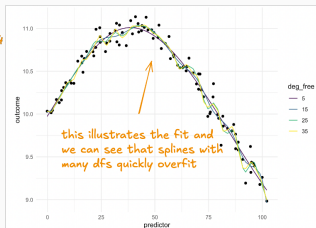
- Instead of creating polynomials on the entire domain, we can also use B-splines (basis splines) which are a collection of piecewise polynomial functions. This makes B-splines more flexible.
- The position where we glue the pieces together are called knots.
- B-splines also provide good options for extrapolation, where high-degree polynomials show high oscillatory behaviour.

Splines

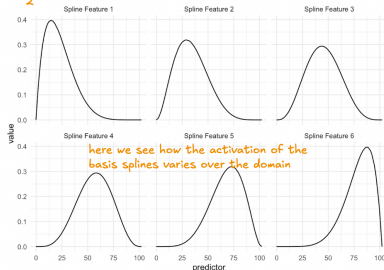
1



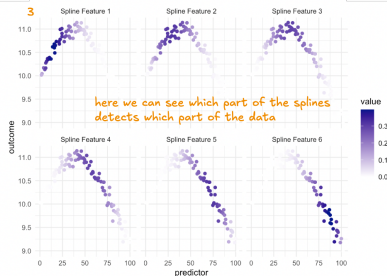
4



2



3



Source: [Example in Feature Engineering A-Z](#)

Splines

```
import pandas as pd
from sklearn.preprocessing import SplineTransformer
```

```
# Create a sample DataFrame with a numeric variable
df = pd.DataFrame({'numeric_feature': [1, 2, 3, 4, 5]})
```

```
# Initialize the SplineTransformer with B-Spline basis
spline = SplineTransformer(n_knots=4, degree=3, include_bias=False, extrapolation="constant")
```

```
# Transform the numeric variable to include B-Spline features
dft = spline.fit_transform(df[['numeric_feature']])
```

```
print(dft)
```

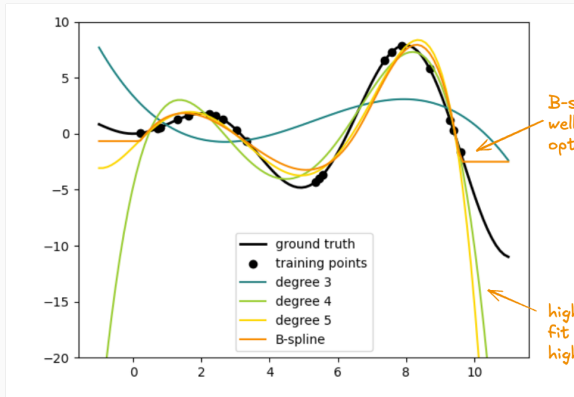
✓ 0.0s

```
[[0.16666667 0.66666667 0.16666667 0.          0.          ]
 [0.00260417 0.31510417 0.61197917 0.0703125  0.          ]
 [0.          0.02083333 0.47916667 0.47916667 0.02083333]
 [0.          0.          0.0703125  0.61197917 0.31510417]
 [0.          0.          0.          0.16666667 0.66666667]]
```

define number of knots and degree of polynomial, as well as extrapolation for values outside of support

observe how the basis functions are piecewise distributed over the domain

Degrees of freedom and extrapolation

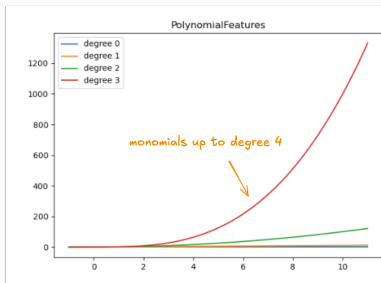


B-splines can also fit the data well and smooth and have better options to control extrapolation

higher order polynomials can fit the data better, but risk of high oscillation and bad extrapolation

Source: [scikit-learn docs](#)

Degrees of freedom and extrapolation



Source: [scikit-learn docs](#)

Here are also some [great slides](#) with more details about splines.

Categorical features



Categorical features

Categorical features are a type of data variable in statistics and machine learning that represent categories or labels. Some shared characteristics:

- **Distinct values:** Categorical features have a finite number of categories. E.g., weekdays, colours, etc.

Categorical features

Categorical features are a type of data variable in statistics and machine learning that represent categories or labels. Some shared characteristics:

- **Distinct values:** Categorical features have a finite number of categories. E.g., weekdays, colours, etc.
- **No inherent order:** Typically there is no inherent order present, yet depending on the context we might want to assume a specific order, e.g. to include monotonicity constraints.

Categorical features

Categorical features are a type of data variable in statistics and machine learning that represent categories or labels. Some shared characteristics:

- **Distinct values:** Categorical features have a finite number of categories. E.g., weekdays, colours, etc.
- **No inherent order:** Typically there is no inherent order present, yet depending on the context we might want to assume a specific order, e.g. to include monotonicity constraints.
- **Encoding:** For most estimators, we have to encode categorical features in numerical values, e.g. by one-hot-encoding. One-hot encoding can lead to a high-dimensional dataset pretty quickly, hence we will also talk about target encoding later on.

Note

A categorical feature does not necessarily have to be a string variable, it can also be a numeric label which we do not aim to use as a numeric feature as this would imply some ranking.

Example: Say, we have a variable denoting the year of some sales data (2021, 2022, 2023, 2024). If we kept it as is and use a linear model, say OLS, we would fit some linear trend, while we might want to capture year fixed effects instead and hence rather one-hot-encode the variable.

Ordinal Encoder

Sometimes we might want to provide some information of the contextual order of a categorical, e.g. to include a monotonicity constraint.

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder

# Define the categorical variable
df = pd.DataFrame({"class": np.random.choice(["basic", "economy", "business", "first class"], size=100)})

# Create an instance of OrdinalEncoder
encoder = OrdinalEncoder(categories=[["basic", "economy", "business", "first class"]])

# Fit and transform the categories
df["encoded_class"] = encoder.fit_transform(df)
```

let's define some categorical variable

here we initiate the OrdinalEncoder with some given order of the categorical

0.0s

	class	encoded_class
0	first class	3.0
1	basic	0.0
2	business	2.0
3	basic	0.0
4	first class	3.0
...	...	...
95	business	2.0
96	basic	0.0
97	economy	1.0
98	business	2.0
99	first class	3.0

it encoded the levels correctly, ranging from "basic" -> 0 to "first class" -> 3. We could use this encoded information to put some monotonicity constraint on the feature as we now gave it an order.

100 rows x 2 columns

One-Hot-Encoder

In the absence of any monotonicity constraint we usually dummy encode the categories, aka one-hot-encoding.

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

# Define the categorical variable
df = pd.DataFrame({"class": np.random.choice(["basic", "economy", "business", "first class"], size=100)})

# Create an instance of OneHotEncoder
encoder = OneHotEncoder(sparse_output=False)

# Fit and transform the categories
encoder.fit(df)

pd.DataFrame(encoder.transform(df), columns=encoder.categories_)
✓ 0.0s
```

here we left the output dense, as the dataframe is quite small, but to save memory, especially for many categories it makes sense to use sparse arrays (the default)

	basic	business	economy	first class
0	0.0	0.0	0.0	1.0
1	0.0	0.0	1.0	0.0
2	0.0	0.0	1.0	0.0
3	0.0	0.0	0.0	1.0
4	1.0	0.0	0.0	0.0
...	...	...	...	...
95	0.0	1.0	0.0	0.0
96	1.0	0.0	0.0	0.0
97	0.0	0.0	0.0	1.0
98	0.0	0.0	1.0	0.0
99	0.0	1.0	0.0	0.0

Now we have dummies for each category. To not fall into the dummy trap we still have to drop one, or we could have set drop="first" for the OneHotEncoder

100 rows x 4 columns

Handling of infrequent or unseen categories

Oftentimes we refrain to estimate individual effects for rare category as we only observed it a few times and the estimate would come with a high degree of uncertainty.

Example

Think about using car brand as feature for used car pricing. You will have many data points for the well known brands but might have very few data points for unknown brands. Instead of potentially overfitting on the few observations, we might bundle them into a residual category of infrequent brands.

Handling of infrequent or unseen categories

This can be achieved by the parameters `min_frequency` and `max_categories`.

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

# Define the categorical variable
df = pd.DataFrame(
    {"car_brand":
     ["VW"]*28 + ["BMW"]*12 + ["Mercedes"]*12 + ["Vauxhall"]*18 +
     ["Ford"]*25 + ["Toyota"]*20 + ["Audi"]*22 + ["BYD"]*3 + ["Lynk&Co"]*2
    })

# Create an instance of OneHotEncoder
encoder = OneHotEncoder(min_frequency=10)

# Fit and transform the categories
encoder.fit_transform(df)

print(encoder.infrequent_categories_)
```

✓ 0.0s

```
[array(['BYD', 'Lynk&Co'], dtype=object)]
```

Handling of infrequent or unseen categories

We also have to decide on what to do in the case of unknown categories at inference. We have the following options for the argument

`handle_unknown`:

- `error`: Throw an error.
- `ignore`: This just adds a column of zeros so the predictions would fall back to the intercept.
- `infrequent_if_exists`: Treat it as infrequent category.

Sometimes, we want even more control over unseen values in productive models and could augment the options by subclassing the respective transformers.

A note on popular Gradient Boosting libraries

The most popular boosting libraries offer support for categoricals:

- The **LightGBM** package (a very flexible package for boosted trees in Python) does not require extra dummy columns/one-hot encoding. It can simply be trained with the categorical variable as integers and a notification which columns are categorical variables.
- **XGBoost** also offers experimental support for categoricals starting from version 1.5.

A note on popular Gradient Boosting libraries

The most popular boosting libraries offer support for categoricals:

- The **LightGBM** package (a very flexible package for boosted trees in Python) does not require extra dummy columns/one-hot encoding. It can simply be trained with the categorical variable as integers and a notification which columns are categorical variables.
- **XGBoost** also offers experimental support for categoricals starting from version 1.5.

Note

The actual splits will however differ when providing the categorical variable as one-hot encoded, as we can make "Yes"/"No" splits vs. an integer encoded categorical, where we separate into groups of categories. The latter might need more splits to arrive at a single category in one node.

Missings



Coping with missing values

A key factor in the effect of missing values is whether data is missing "at random" or systematically. When data is missing for a reason (as is usually the case in the real world), then the fact that it was missing is an important piece of information that we should not throw away.

Coping with missing values

A key factor in the effect of missing values is whether data is missing "at random" or systematically. When data is missing for a reason (as is usually the case in the real world), then the fact that it was missing is an important piece of information that we should not throw away.

Our preferred recipe for dealing with missing values usually is:

- If the variable is numerical:
 1. Impute with a sensible value (e.g. the mean or median)
 2. Add a binary indicator that flags whether the variable was imputed
- If the variable is categorical:

Create a new level for the categorical variable called "missing"

What does research say about simple imputation strategies?

Bertsimas et al. (2024) study the performance of various "impute-then-regress" pipelines and provide support for the simple imputation rules.

They show that:

- Mean imputation is asymptotically optimal.
- Mode imputation is sub-optimal.
- Missing categoricals should be encoded outside of the support as "missing".

So how can we impute then?

We can use scikit-learn's `SimpleImputer` which offers various imputing strategies. There is also an illustrative example [here](#), comparing various strategies.

```
import numpy as np
from sklearn.impute import SimpleImputer

# Creating some sample data with missing values
X = np.array([[1, 2],
              [np.nan, 3],
              [4, np.nan],
              [5, 6]])

# Creating the SimpleImputer object
imputer = SimpleImputer(strategy='mean', add_indicator=True)

# Fitting the imputer on the data
imputer.fit(X)

# Transforming the data
X_imputed = imputer.transform(X)

# Displaying the imputed data
print("Imputed Data:")
print(X_imputed)
```

Mean imputation, while also adding a missing indicator

Data was successfully imputed and indicators for both columns were added

Imputed Data:

[1.	2.	0.	0.	]
[3.33333333	3.	1.	0.	]
[4.	3.66666667	0.	1.	]
[5.	6.	0.	0.	]

Outliers



Outliers

Outliers can occur in features as well as in target variables. For treating outliers adequately it is very beneficial to understand their background story. For instance:

- Is it just due to a typo that the observation is magnitudes larger than others?
- Is such an outlier possible giving the underlying data generating process (e.g. very large insurance claims for rare events)?

In the first case we might want to drop or winsorize the data, while in the second case we are interested in capturing such outliers in the best possible way.

Note

We will talk more about the impact of outliers in the dependent variable on the loss function and how to specifically model outliers in the modelling chapter.

Note

Scikit-learn does not offer an implementation for simple winsorization. We could either write our own transformer, as we will learn later today, or we can resort to [feature engine](#) , a designated feature engineering library following scikit-learn's API with plenty of extra transformers.

Winsorize outliers

```
import numpy as np
import pandas as pd
from feature_engine.outliers import Winsorizer
```

Initialize Winsorizer, using quantile capping, where 'fold' refers to the quantile to be capped, here on both sides.

```
# Creating a DataFrame with normally distributed data
data = pd.DataFrame({'A': np.random.normal(0, 1, 1000)})
```

```
# Creating the Winsorizer object with quantile capping method
winsorizer = Winsorizer(capping_method='quantiles', tail='both', fold=0.05)
```

```
# Fitting the Winsorizer on the data
winsorizer.fit(data)
```

```
# Transforming the data
data_winsorized = winsorizer.transform(data)
```

```
# Displaying the winsorized data
print("Winsorized Data:")
data_winsorized.value_counts()
```

✓ 0.0s

Winsorized Data:

A	
-1.629318	50
1.660198	50
-1.615070	1
0.257615	1
0.338830	1

The transformed data correctly shows 5% winsorized observations at each tail.

Dimension reduction



Unsupervised dimension reduction - PCA

If we have many numerical and potentially highly correlated variables we could run a Principal Component Analysis (PCA) to extract as much variance as possible by as little features as possible.

Example

Let's consider again the used car pricing example. There we have many numerical and correlated variables describing a car:

- Size: length, height, width, cubic capacity, number of seats, weight,...
- Motor: horse powers, acceleration, max speed, emissions...
- Usage: age, mileage,...

Running a PCA could help to identify clusters of specific car types.

Illustration PCA - Iris dataset

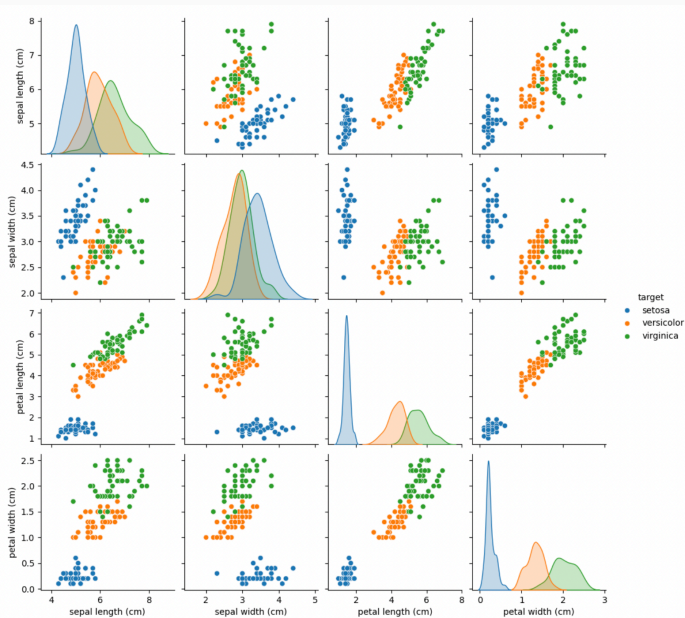
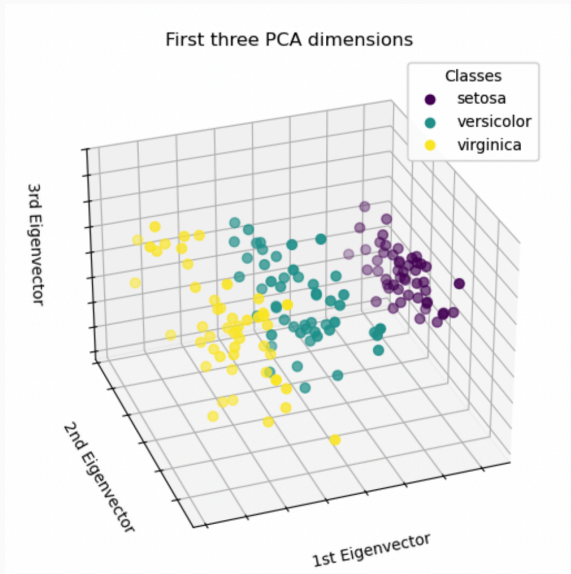


Illustration PCA - Iris dataset



Supervised dimension reduction - Target encoding

If you have very high dimensional categorical variables on the RHS of the equation (e.g. 200 categories), a more memory efficient option than having hundreds of dummy columns is to use so-called target encoding. Rather than creating as many dummy variable columns as categorical classes, only one variable is created. This variable contains the group mean (or another summary statistic) for each level of the category.

Example

Going back to our used car pricing example, one could think about target encoding the car brands.

Target encoding

```
import string
import pandas as pd
import numpy as np
from category_encoders.target_encoder import TargetEncoder

# Sample data for demonstration
np.random.seed(42)
data = {
    'category': np.random.choice([letter for letter in string.ascii_lowercase], size=100),
    'target': np.random.uniform(size=100)
}
df = pd.DataFrame(data)

# Initialize TargetEncoder
encoder = TargetEncoder(smoothing=0, min_samples_leaf=0)

# Fit and transform the target encoding on the 'category' column
encoded_category = encoder.fit_transform(df['category'], df['target'])

# Adding the encoded column back to the DataFrame
df['category_encoded'] = encoded_category

# Calculate actual target means for each category
actual_means = df.groupby('category')['target'].mean()

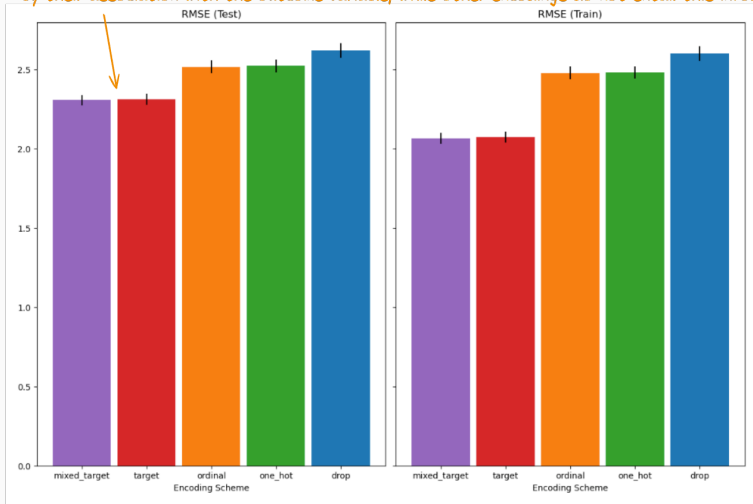
# Check if the computed means match the actual group means
computed_means = df.groupby('category')['category_encoded'].mean()

# Compare actual means with computed means
assert all(np.isclose(actual_means, computed_means))
```

using smoothing weights and min_samples_leaf
we can regularize the target encoding, here we choose
no regularization to compare it with the actual means

Target encoding

target encoding can lead to performance gains as high-dimensional categories get ordered by their association with the outcome variable, while other encodings do not entail this information.



Source: [scikit-learn example](#)

Extracting features from text



Extracting features from text

Sometimes we might want to leverage information provided as text, e.g. details about some insurance claim. Depending on the extraction method used, this can also be seen as dimension reduction, as using the entire string with all its tokens can become very high-dimensional relatively quickly. Hence, we want to extract specific commonalities which we then could use in a subsequent model.

Modern LLMs could also be used as a first stage to provide high-quality signals from the text.

Simple example using CountVectorizer

```
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd
```

```
# Sample text data - movie reviews
```

```
data = {
    'text': [
        'This movie is great and wonderful',
        'I did not like the acting in this film',
        'The plot of the movie was predictable but the acting was good'
    ]
}
```

some data containing
column(s) with text

```
# Create a DataFrame from the text data
```

```
df = pd.DataFrame(data)
```

```
# Extract features from the text using CountVectorizer
```

```
count_vectorizer = CountVectorizer()
```

```
X_text_features = count_vectorizer.fit_transform(df['text'])
```

CountVectorizer converts text
to matrix of token counts

```
# Display the extracted features as a DataFrame
```

```
df_text_features = pd.DataFrame(X_text_features.toarray(), columns=count_vectorizer.get_feature_names_out())
df_text_features
```

✓ 0.0s

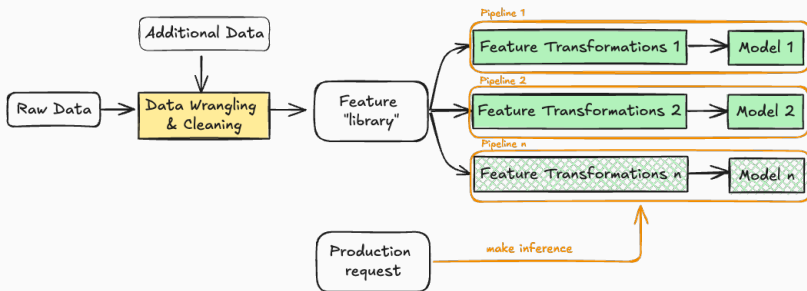
	acting	and	but	did	film	good	great	in	is	like	movie	not	of	plot	predictable	the	this	was	wonderful
0	0	1	0	0	0	0	1	0	1	0	1	0	0	0	0	0	1	0	1
1	1	0	0	1	1	0	0	1	0	1	0	1	0	0	0	1	1	0	0
2	1	0	1	0	0	1	0	0	0	0	1	0	1	1	1	3	0	2	0

vectorized text data

Feature Transformation Pipelines



Feature Transformation Pipelines



Note

Using pipelines allows us to glue the specific feature transformations to the respective model, making it easier to port them to production.

Feature Transformation Pipelines

Scikit-learn's `Pipeline` class allows us to chain various feature transformations before including an optional estimator (more on the latter in the next chapter).

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
steps = [
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components=2))
]
```

```
numeric_pipeline = Pipeline(steps)
```

define sequential
transformations

initialise Pipeline
with transform
steps

Feature Transformation Pipelines - The ColumnTransformer

In the simple example we would use the same sequential transformations on all features, however, we are usually interested in feature specific transformations. Scikit-learn's **ColumnTransformer** helps us here.

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```
data = {
    'numeric_feature': [1, 4, 7],
    'categorical_feature': ['A', 'B', 'C']
}
```

```
df = pd.DataFrame(data)
```

```
numeric_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())
])
```

individual transform
pipelines for specific
features

```
categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder())
])
```

```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, ['numeric_feature']),
        ('cat', categorical_transformer, ['categorical_feature'])
    ]
)
```

ColumnTransformer
assigns specific
transformers to columns

```
full_pipeline = Pipeline(steps=[('preprocessor', preprocessor)])
```

note how we can
nest multiple
Pipelines

Writing your own transformer

- Scikit-learn implements many transformers for you, but being able to write your own custom transformer is really helpful as it gives you a lot of flexibility.
- Example use case: An example could be that the model should be served from different data dialects and we need a translation transformer.
- Why should we follow the scikit-learn API if it's custom anyway? By conforming with the API our transformer integrates seamlessly and can be chained with other transformers.

Writing your own transformer

Ingredients:

- Class which inherits from `BaseEstimator` and `TransformerMixin`.
- Should implement 'fit', 'transform' and (optional) 'fit_transform' method.

```
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin

class SquareTransformer(BaseEstimator, TransformerMixin):
    def __init__(self, columns):
        self.columns = columns

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X_transformed = X.copy()
        for col in self.columns:
            if col in X_transformed.columns and pd.api.types.is_numeric_dtype(X_transformed[col]):
                X_transformed[col] = X_transformed[col] ** 2
        return X_transformed
```

inherit from scikit-learn's base transformer classes

fit is just a pass in this case, as we don't have to learn anything from the data in this particular case

the transform method implements the squaring of columns and adds some validation that they are numeric

Writing your own transformer - Let's check that it worked

```
# Sample DataFrame with numeric and non-numeric columns
```

```
data = {
```

```
    'A': [2, 4, 6],
```

```
    'B': [3, 5, 7],
```

```
    'C': ['apple', 'banana', 'cherry']
```

```
}
```

```
df = pd.DataFrame(data)
```

```
# Instantiate and apply the custom transformer to square numeric columns 'A' and 'B'
```

```
squared_transformer = SquareTransformer(columns=['A', 'B'])
```

```
transformed_data = squared_transformer.fit_transform(df)
```

✓ 0.0s

```
transformed_data
```

✓ 0.0s

	A	B	C
0	4	9	apple
1	16	25	banana
2	36	49	cherry

↑
we provide columns to be squared
and run fit_transform (or transform)
to compute the squares

Writing your own transformer - Let's test that it worked

```
import pytest
import pandas as pd
from fun_ds.preprocessing import SquareTransformer

@pytest.fixture
def df_in():
    return pd.DataFrame(
        {
            "A": [-1, 1, 2, 3, 4, 5, 6, 7, 8, 9],
            "B": ["a", "b", "c", "d", "e", "f", "g", "h", "i", "j"]
        }
    )

@pytest.fixture
def df_out():
    return pd.DataFrame(
        {
            "A": [1, 1, 4, 9, 16, 25, 36, 49, 64, 81],
            "B": ["a", "b", "c", "d", "e", "f", "g", "h", "i", "j"]
        }
    )

@pytest.mark.parametrize("columns", [{"A"}, {"A", "B"}])
def test_square_transformer(df_in, df_out, columns):
    df = SquareTransformer(columns=columns).transform(df_in)
    pd.testing.assert_frame_equal(df, df_out)
```

we define all inputs
we need for the test
as pytest fixtures

for a unit test, we need
some expected output

we can parametrize the test
to test various configurations

finally, we want to assert that
the output matches our expectation

Recommended resources

I just recently came across this open-source online book: [Feature Engineering A-Z](#), which appears to be a very comprehensive overview with R and Python examples, yet some parts are still "under construction".